



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.



SDD

System Design Document

Il Faro dello Studio

Versione	2.0
Data	04/01/2026
Destinatario	Prof. Carmine Gravino
Presentato da	Gruppo NC15: Alfano Daniele, Ambrunzo Aniello Cristiano, Amoroso Salvatore, Della Monica Cristian
Approvato da	



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

Revision History

Data	Versione	Descrizione	Autori
10/11/2025	0.1	Prima stesura	Tutto il team
13/11/2025	0.2	Analisi dei requisiti non funzionali e stesura dei Design Goals.	AD, AA
17/11/2025	0.3	Revisione dei Design Goals	Tutto il team
20/11/2025	0.4	Decomposizione in sottosistemi e aggiunta del diagramma di deployment.	AS, DMC
22/11/2025	0.5	Definizione delle condizioni limite	AS, AA
24/11/2025	0.6	Revisione dei sottosistemi	Tutto il team
26/11/2025	0.7	Stesura della sezione "Gestione dei dati persistenti" (aggiunta Class Diagram e DB)	AD, DMC
27/11/2025	0.8	Stesura della sezione "Controllo globale degli accessi" e aggiunta dei servizi dei sottosistemi	AS, AA
28/11/2025	0.9	Definizione dei due Design Patterns	AD, DMC
01/12/2025	1.0	Revisione finale e stesura del glossario.	Tutto il team
02/01/2026	1.1	Revisione Gestione Dati Persistenti: Verifica	Tutto il team



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

		consistenza e allineamento dettagli tra Class Diagram e Modello ER.	
04/01/2026	1.2	Revisione e correzione paragrafo Design Patterns.	AS
04/01/2026	2.0	Revisione finale	Tutto il team

Team Members

Nome	Acronimo	Contatto
Alfano Daniele	AD	d.alfano29@studenti.unisa.it
Amoroso Salvatore	AS	s.amoroso11@studenti.unisa.it
Ambrunzo Aniello Cristiano	AA	a.ambrunzo@studenti.unisa.it
Della Monica Cristian	DMC	c.dellamonica18@studenti.unisa.it



Sommario

1 Introduzione

- 1.1 Scopo del sistema
- 1.2 Obiettivi di Design (Design Goals)
- 1.3 Definizioni, acronimi, e abbreviazioni
- 1.4 Riferimenti
- 1.5 Organizzazione del documento

2 Architettura software proposta

- 2.1 Panoramica sulla sezione
- 2.2 Decomposizione in sottosistemi
 - GUI (Graphic User Interface)
 - Controller
 - DAO (Data Access Object)
- 2.3 Mapping Hardware/Software
- 2.4 Gestione dei dati persistenti
- 2.5 Controllo degli accessi e sicurezza
- 2.6 Controllo globale del software
- 2.7 Condizioni limite
- 2.8 Design Patterns

3 Servizi dei sottosistemi

4 Glossario



1 Introduzione

1.1 Scopo del sistema

Il Faro dello Studio si propone di facilitare e potenziare la comunicazione e l'organizzazione delle attività didattiche all'interno dell'ambiente di un doposcuola, offrendo uno strumento digitale che avvicina studenti, famiglie e docenti in un'unica piattaforma interattiva.

Il sistema, gestito da uno o più Amministratori, permette la registrazione e l'accesso delle diverse tipologie di utenti, ognuna con funzionalità dedicate, e consente ai docenti di creare e gestire attività didattiche, lezioni ed eventi formativi in modo semplice e strutturato.

Elemento centrale del sistema è il Calendario Didattico, che permette a studenti di visualizzare le attività disponibili e iscriversi a quelle di proprio interesse.

Accanto a questo, il sistema integra funzionalità per la gestione dei pagamenti scolastici, tramite cui le famiglie possono visualizzare, effettuare e consultare lo storico dei propri versamenti.

Per favorire il dialogo e la qualità della proposta formativa, *Il Faro dello Studio* offre inoltre la possibilità di inserire feedback sui docenti e sulle attività svolte, contribuendo a valorizzare il percorso educativo e a migliorare i servizi offerti dalla scuola.

1.2 Obiettivi di Design (Design Goals)

Nella presente sezione si andranno a presentare i Design Goals, ovvero le qualità sulle quali il sistema deve essere focalizzato, formalizzati esplicitamente così che qualsiasi importante decisione di design può essere fatta consistentemente seguendo lo stesso insieme di design goal.

Seguendo le linee guida del libro Bernd Bruegge – Object Oriented Software Engineering i design goal sono stati suddivisi nelle seguenti categorie:

- **Performance:** includono i requisiti di spazio e velocità imposti sul sistema.



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

- **Dependability:** determinano quanto sforzo deve essere speso per minimizzare i fallimenti del sistema (crash, falle di sicurezza) e le loro conseguenze.
- **Maintenance:** determina quanto sforzo è necessario per modificare il sistema dopo il suo rilascio.
- **End User:** includono qualità che sono desiderabili dal punto di vista dell'utente, ma che non sono state coperte dai criteri di Performance e Dependability.

Ciascun design goal è descritto da:

- **Rank**, che ne specifica un valore di priorità compreso tra 1 e 10 (1 massima e 10 minima).
- **ID Design Goal**, un identificatore univoco e un nome esplicativo.
- **Descrizione**, una descrizione del design goal.
- **Categoria**, ovvero la categoria di appartenenza del design goal.
- **RNF di origine**, ovvero il requisito non funzionale che lo ha generato.

Design goals

Rank	ID Design Goal	Descrizione	Categoria	RNF di origine
2	DG_1 Affidabilità delle Operazioni	Il sistema deve garantire che tutte le operazioni avvengono con successo.	Dependability	RNF_A_1
6	DG_2 Facilità d'Utilizzo	Il sistema deve risultare facilmente comprensibile ed utilizzabile anche da un'utenza meno esperta.	End User	RNF_U_1
5	DG_3 Navigazione concorrente	Il sistema dovrà essere correttamente funzionante anche con un elevato numero di utenti connessi in contemporanea.	Performance	RNF_P_2
9	DG_4 Fallimento di sistema	Il sistema deve sapersi comportare in situazioni di fallimento notificando l'utente, tramite appositi	Dependability	RNF_A_1



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

		messaggi.		
3	DG_5 Gestione permessi	Il sistema deve garantire una divisione tra le varie categorie di utenti, al fine di limitare le funzionalità accessibili ad ognuno e mantenere così l'integrità del sistema.	Dependability	RNF_A_2
10	DG_6 Disponibilità del sistema	Il Sistema deve garantire la massima disponibilità, con un limite di 48 ore all'anno di downtime.	Dependability	RNF_P_1
1	DG_7 Sicurezza dei dati	Il Sistema deve garantire la massima sicurezza dei dati, utilizzando protocolli di comunicazione sicuri, e assicurando la visualizzazione dei dati solo agli utenti che hanno diritto ad accedervi.	Dependability	RNF_S_1
4	DG_8 Manutenibilità	Il sistema deve essere facilmente manutenibile ed estendibile.	Maintenance	RNF_SM_2
8	DG_9 Estendibilità	Il sistema si presta facilmente all'aggiunta di nuove funzionalità date le elevate necessità dell'utenza.	Maintenance	RNF_SM_1
7	DG_10 Interfaccia intuitiva	L'interfaccia utente della piattaforma deve permettere di eseguire azioni in modo chiaro e semplice, rendendo ben esplicita la funzionalità di ogni elemento visuale.	End User	RNF_U_2



Trade-off

Trade-off	Descrizione
Sicurezza vs Facilità d'utilizzo	Garantire elevati livelli di sicurezza richiede l'adozione di controlli aggiuntivi e procedure più rigide, rendendo meno immediata l'interazione per gli utenti meno esperti; occorre quindi bilanciare protezione dei dati e semplicità d'utilizzo.
Prestazioni vs Affidabilità	Meccanismi di controllo e validazione aumentano l'affidabilità delle operazioni ma introducono overhead che può rallentare il sistema in caso di accessi concorrenti; è necessario trovare un equilibrio tra rapidità di risposta e robustezza delle funzionalità.

1.3 Definizioni, acronimi, e abbreviazioni

Vengono riportati di seguito alcune definizioni presenti nel documento corrente:

- **Sottosistema:** un sottoinsieme dei servizi del dominio applicativo, formato da servizi legati da una relazione funzionale.
- **Design Goal (DG):** le qualità sulle quali il sistema deve essere focalizzato.
- **Mapping Hardware/Software:** studio della connessione tra parti fisiche e logiche di cui si compongono il sistema.
- **Gestione dati persistenti:** dati che sopravvivono all'esecuzione del programma che li ha creati e che dunque vengono salvati.
- **SDD:** System Design Document
- **RAD:** Requirements Analysis Document

1.4 Riferimenti

Di seguito una lista di riferimenti ad altri documenti utili durante la lettura:

- Requirements Analysis Document;
- System Design Document;
- Test Plan;



1.5 Organizzazione del documento

Il presente documento di System Design consta di tre sezioni:

Introduzione: Viene descritto in generale lo scopo del sistema, gli obiettivi di design che il sistema propone di raggiungere.

Architettura software proposta: Viene descritto come il sistema sarà definito e partizionato in sottosistemi, il loro mapping Hardware/Software, la gestione dei dati persistenti. Verranno poi presentate la struttura dei singoli sottosistemi e le boundary conditions riguardanti l'intero sistema.

Glossario: Contiene la lista dei termini usati nel documento con annessa spiegazione.

2 Architettura software proposta

2.1 Panoramica sulla sezione

Il sistema proposto adotta uno stile architettuale **Three Tier**. Questa architettura si adatta perfettamente alle esigenze del progetto *Il Faro dello Studio*, poiché garantisce una netta separazione tra la logica di presentazione, la logica applicativa e la gestione dei dati, migliorando le seguenti qualità del software:

- **Leggibilità**, grazie alla chiara distinzione tra le responsabilità dei vari componenti;
- **Manutenibilità**, poiché eventuali modifiche sono isolate all'interno del livello corrispondente;
- **Estendibilità**, rendendo semplice l'aggiunta di nuove funzionalità nella piattaforma;
- **Riusabilità**, facilitata dalla presenza di entità e controller riutilizzabili in diversi contesti.



Per la parte di **front-end**, il sistema utilizzerà **HTML5**, **CSS3** e **JavaScript**, impiegati per la definizione delle interfacce utente e la gestione dell'interazione con i vari componenti del sistema.

La logica di **back-end** verrà sviluppata in **Java**.

Per quanto riguarda la gestione dei dati, il sistema sfrutterà **MySQL** come database relazionale, progettato tramite **MySQL Workbench**.

L'accesso al database sarà centralizzato: un solo membro del team gestirà la struttura del database e la produzione degli script SQL, al fine di garantire coerenza e uniformità tra le varie fasi di sviluppo.

Questa combinazione di architettura e tecnologie fornisce una base solida e affidabile per lo sviluppo del sistema, consentendo una chiara organizzazione interna e una facile evoluzione nelle release future.

2.2 Decomposizione in sottosistemi

Autenticazione: si occupa delle funzioni riguardanti la sicurezza degli accessi. È responsabile dell'identificazione degli attori e dei loro permessi.

Gestione Utenze: Si occupa della registrazione e manutenzione dei profili utente. Permette agli ospiti di registrarsi, agli utenti registrati di visualizzare e modificare i propri dati personali e consente all'Amministratore di intervenire sullo stato degli account (attivazione/disattivazione) per scopi di moderazione.

Gestione Attività: Coordina l'intero ciclo di vita delle attività didattiche. Gestisce la creazione, modifica ed eliminazione delle lezioni da parte dei docenti, la visualizzazione del calendario e il processo di iscrizione degli studenti, verificando la disponibilità dei posti e l'avvenuto pagamento.



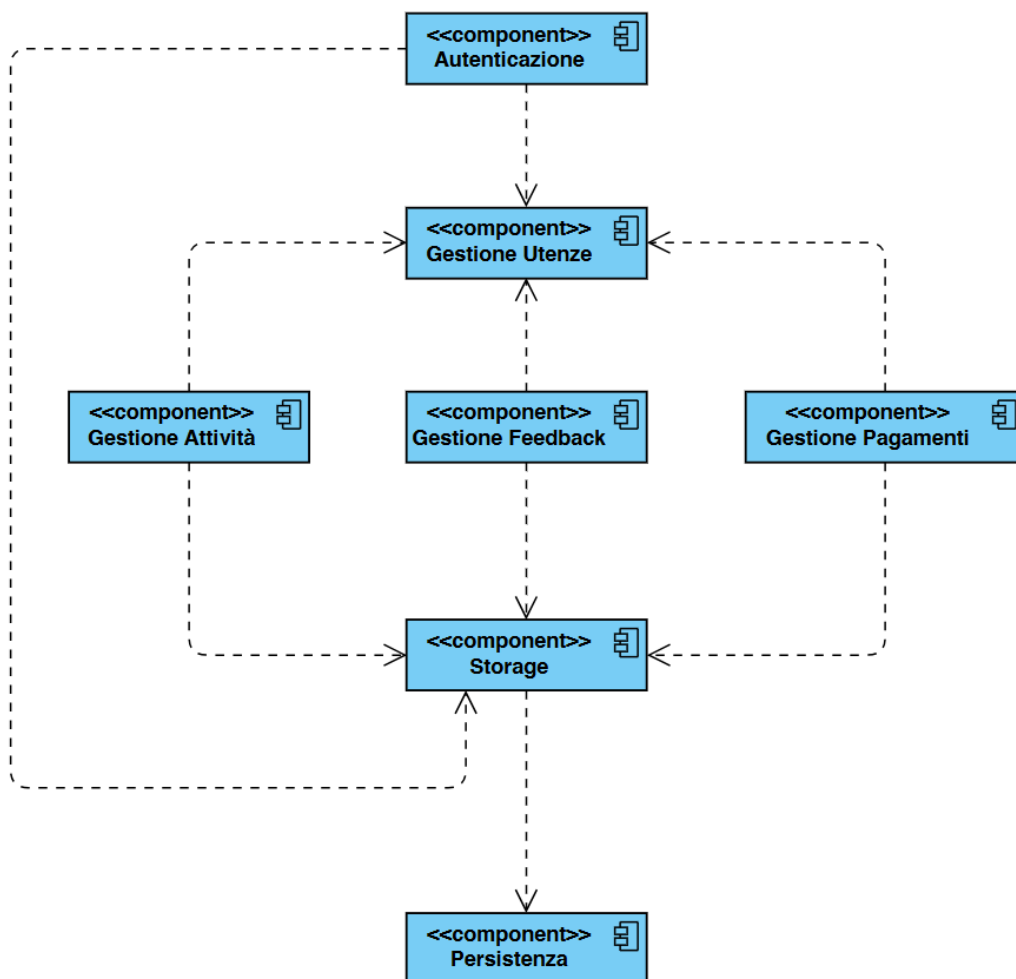
Gestione Feedback: Si occupa delle funzioni riguardanti l'aggiunta di un feedback ad un docente da parte di una famiglia/studente.

Gestione Pagamenti: Si occupa delle funzioni riguardanti i pagamenti effettuati dalle famiglie.

Persistenza: si occupa di gestire la persistenza dei dati con un database.

Storage: si interpone tra i vari sottosistemi e il sottosistema di persistenza.

Sono mostrate di seguito le dipendenze tra i sottosistemi attraverso un component diagram UML.





Alcuni dei sottosistemi del sistema saranno realizzati utilizzando componenti software già esistenti (COTS – *Commercial Off The Shelf*), al fine di semplificare lo sviluppo e garantire maggiore stabilità.

Di seguito l'elenco delle principali componenti selezionate:

- **Gestione della persistenza:** affidata a un DBMS relazionale **MySQL**, progettato tramite *MySQL Workbench* e mantenuto da un singolo membro del team per assicurare uniformità della struttura dati.
- **Interazione con il database:** realizzata tramite un livello **DAO (Data Access Object)**, che incapsula tutte le operazioni di accesso e manipolazione dei dati persistenti.

Di seguito una vista dettagliata dei sottosistemi principali e delle rispettive responsabilità:

- **GUI (Graphic User Interface)**

Contiene tutte le pagine e le componenti di interfaccia che verranno presentate all'utente.

- **Controller**

Si occupa della logica per il controllo del sistema.

- **Service**

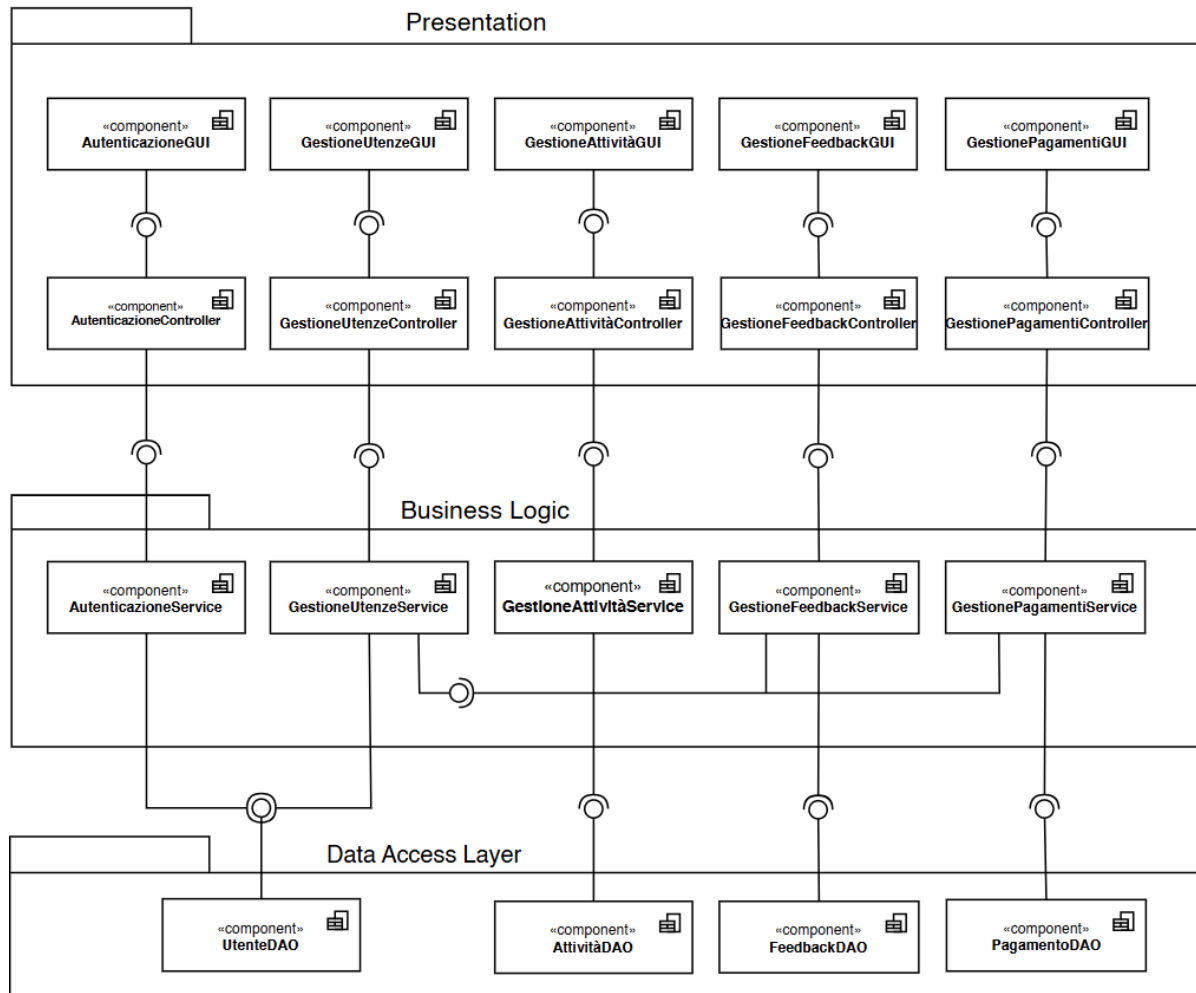
Si occupa della logica di business.

- **DAO (Data Access Object)**

Si occupa di fornire accesso ai dati persistenti

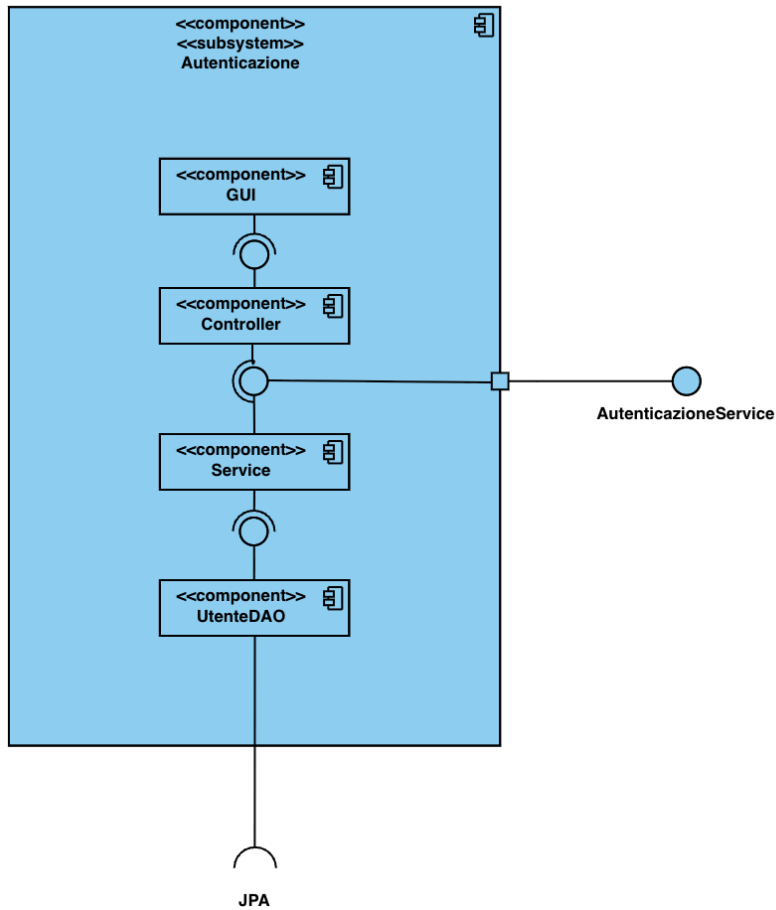


Diagramma architetturale



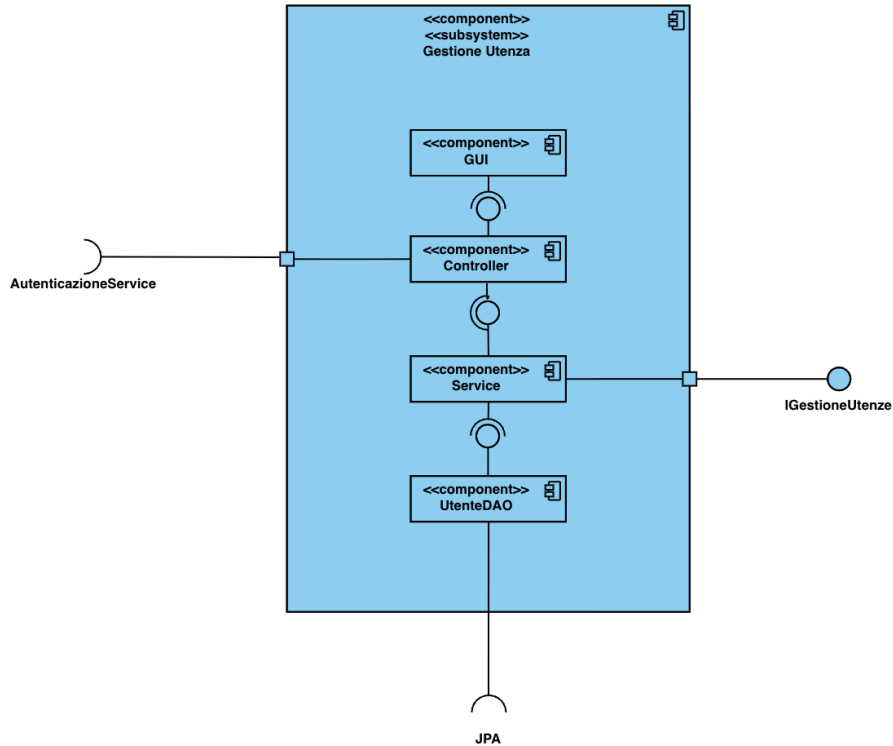


Sottosistema Autenticazione

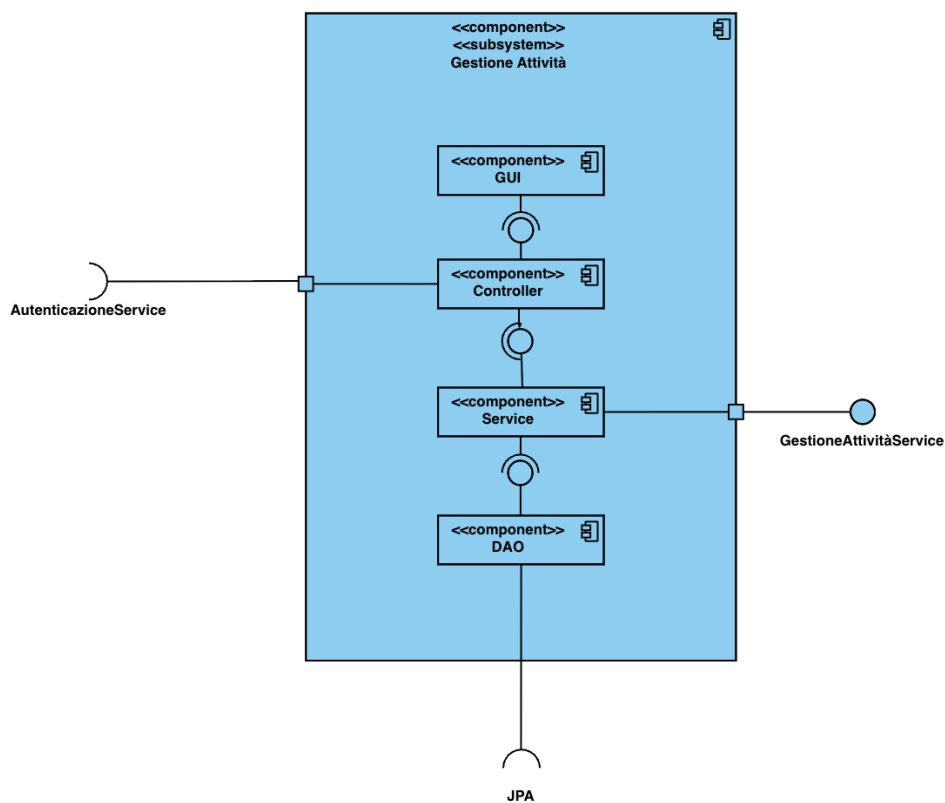




Sottosistema Gestione Utenze

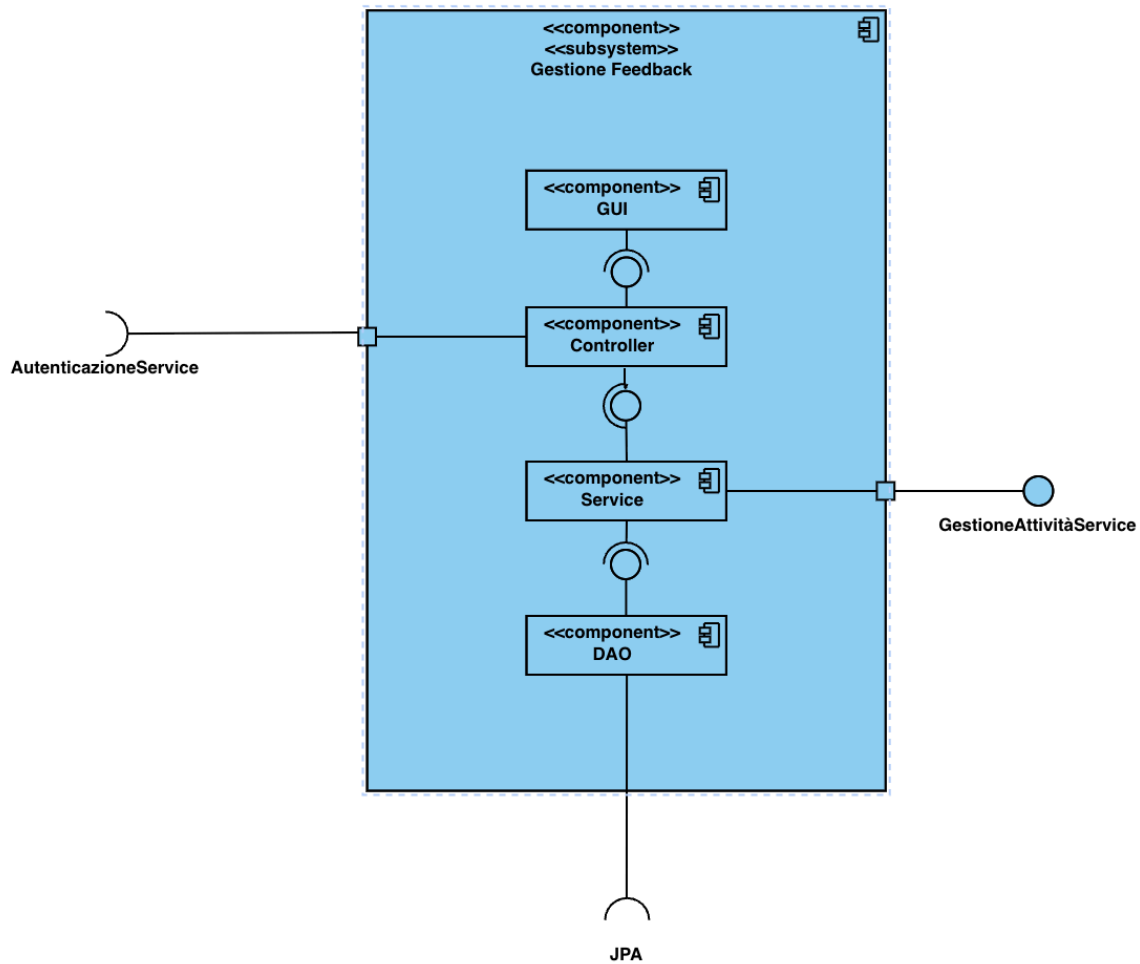


Sottosistema Gestione Attività



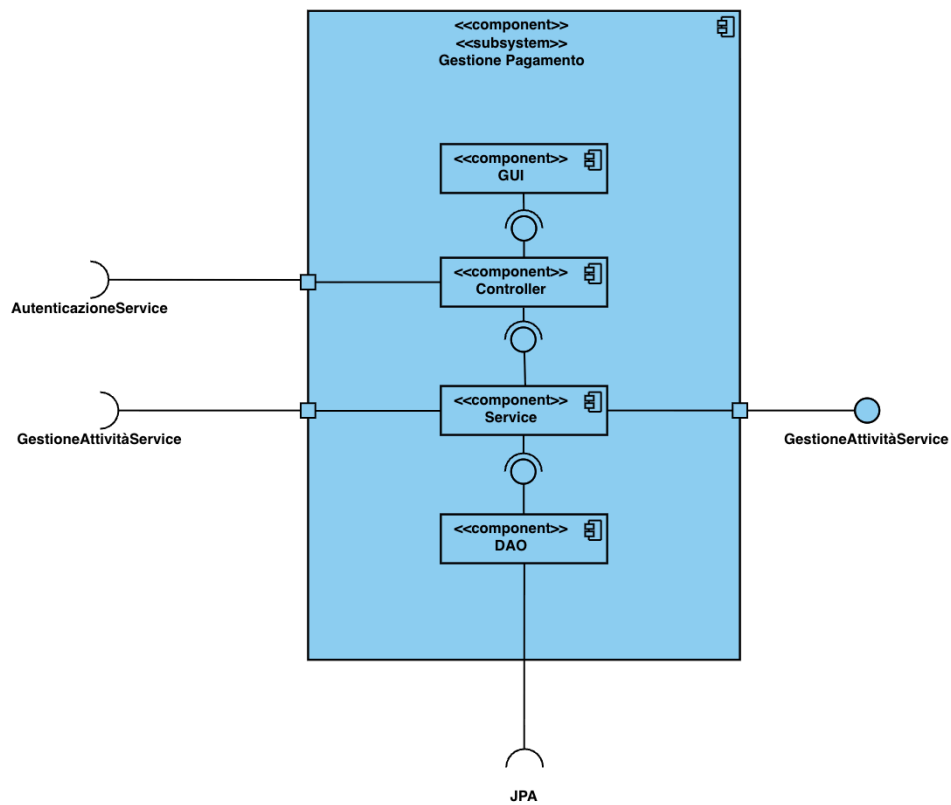


Sottosistema Gestione Feedback





Sottosistema Gestione Pagamenti



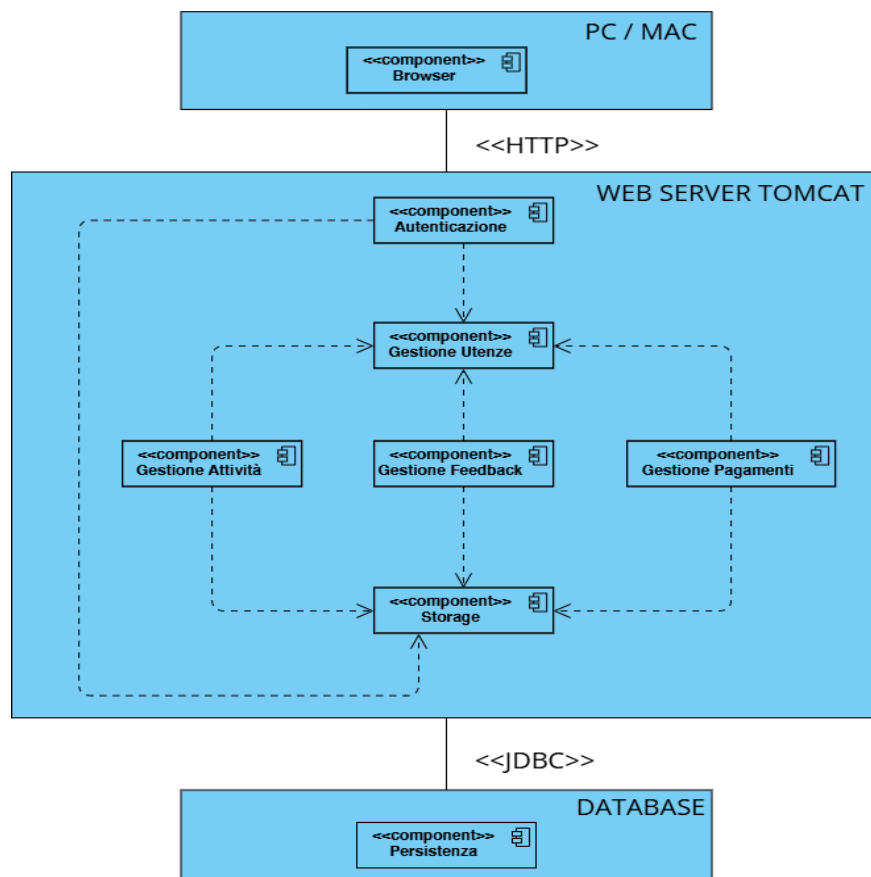


2.3 Mapping Hardware/Software

L'applicazione web sviluppata si basa su un'architettura hardware composta da un **unico server**, incaricato di gestire le richieste provenienti dai client. Gli utenti possono accedere al sistema tramite qualsiasi dispositivo dotato di browser e connessione a Internet.

Poiché *Il Faro dello Studio* è una web application che risiede su un unico web server, la sua architettura non è distribuita e l'intero sistema viene eseguito su un solo nodo elaborativo. Questo modello semplifica la gestione dell'applicazione e riduce la complessità dell'infrastruttura necessaria per il suo funzionamento.

Di seguito viene riportato un **UML Deployment Diagram** che descrive il mapping tra i componenti hardware e i moduli software dell'applicazione.





2.4 Gestione dei dati persistenti

Introduzione

Per la gestione dei dati persistenti del sistema si è scelto di adottare un **database relazionale**, in quanto soluzione più adatta a garantire accesso concorrente controllato e mantenere un elevato livello di consistenza delle informazioni, grazie al supporto offerto da un DBMS.

Questa scelta è pienamente coerente con i design goals stabiliti per il progetto, poiché l'utilizzo di un DBMS permette di beneficiare delle seguenti proprietà:

- **Vincoli di integrità:** un DBMS consente di definire e far rispettare diversi vincoli (chiavi primarie, chiavi esterne, unicità, domini), assicurando che i dati rimangano validi e consistenti in ogni operazione di aggiornamento.
- **Privatezza dei dati:** l'accesso protetto fornito dal DBMS permette di differenziare permessi e privilegi tra utenti, garantendo che ciascun utilizzatore possa visualizzare e modificare solo le informazioni per cui è autorizzato.
- **Affidabilità:** un DBMS offre meccanismi di backup, ripristino e gestione dei guasti, che permettono di recuperare lo stato della base di dati anche in caso di errori software o problemi hardware, aumentando la robustezza del sistema.
- **Atomicità delle operazioni:** grazie al concetto di transazione, il DBMS assicura che una sequenza di operazioni venga eseguita completamente oppure non venga applicata affatto. Ciò consente di preservare la coerenza della base di dati con la realtà modellata, soprattutto nelle operazioni critiche come pagamenti, modifiche di attività e aggiornamento delle iscrizioni.



Queste caratteristiche rendono l'utilizzo di un database relazionale una scelta solida ed efficace per supportare le funzionalità del sistema e garantirne l'affidabilità nel tempo.

Data la necessità di mantenere un ambiente di sviluppo stabile e coerente, la gestione del database è stata affidata a un **unico membro del team**, incaricato di curarne la progettazione, la creazione e gli aggiornamenti strutturali.

Questa decisione è stata presa per evitare i problemi tipici legati alla duplicazione delle basi di dati locali su dispositivi differenti, come incoerenze nella struttura, versioni disallineate o configurazioni non uniformi tra i vari sviluppatori.

Centralizzando la responsabilità del database in un solo referente, il team ha potuto:

- garantire un'unica versione ufficiale e sempre aggiornata dello schema dei dati;
- ridurre il rischio di conflitti o errori dovuti a modifiche parallele non coordinate;
- semplificare il processo di integrazione tra le diverse componenti del sistema;
- diminuire il tempo necessario a risolvere malfunzionamenti derivanti da installazioni locali non omogenee.

Questa scelta organizzativa si è quindi rivelata efficace nel rendere più fluido il lavoro del gruppo e nel garantire un'evoluzione controllata e affidabile della base di dati durante tutto il ciclo di sviluppo.

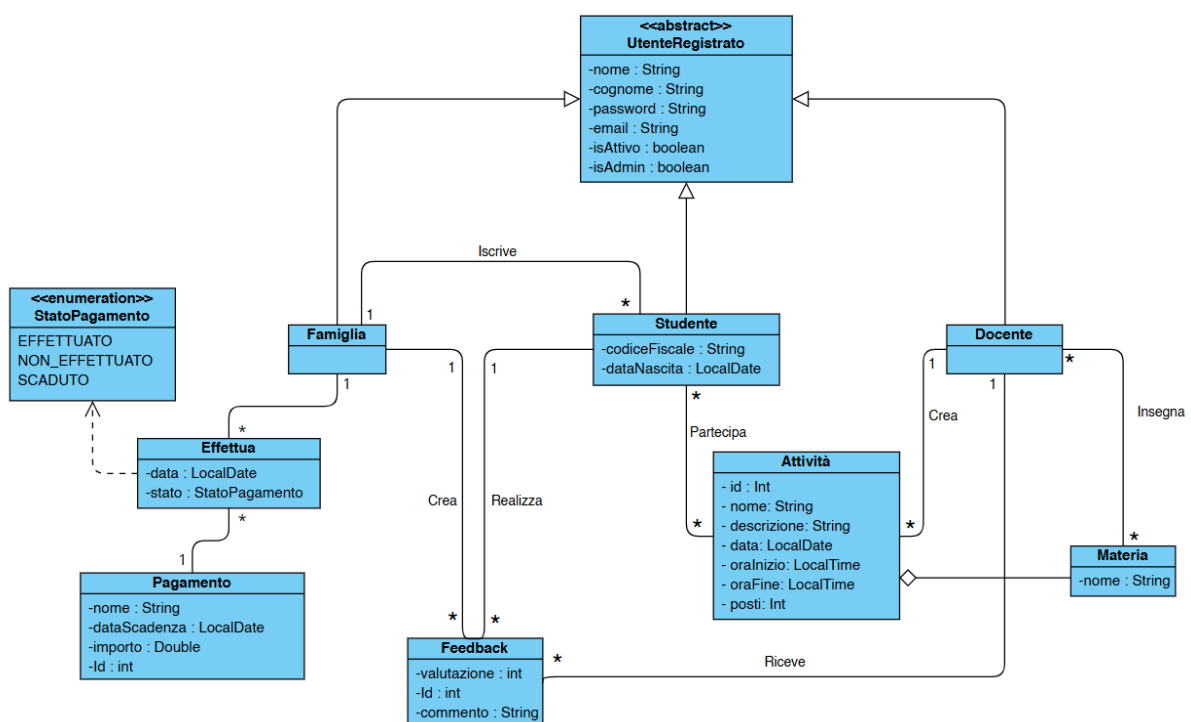


CD_SDD: Entity Diagram Ristrutturato

Durante la fase di Object Design, il Class Diagram relativo alle entity del sistema non ha subito variazioni sostanziali rispetto alla versione definita nel RAD. Dopo un'attenta analisi dei requisiti e delle funzionalità previste per la prima release del progetto, è stato infatti ritenuto che la modellazione iniziale fosse già adeguata a rappresentare in modo completo e coerente il dominio applicativo.

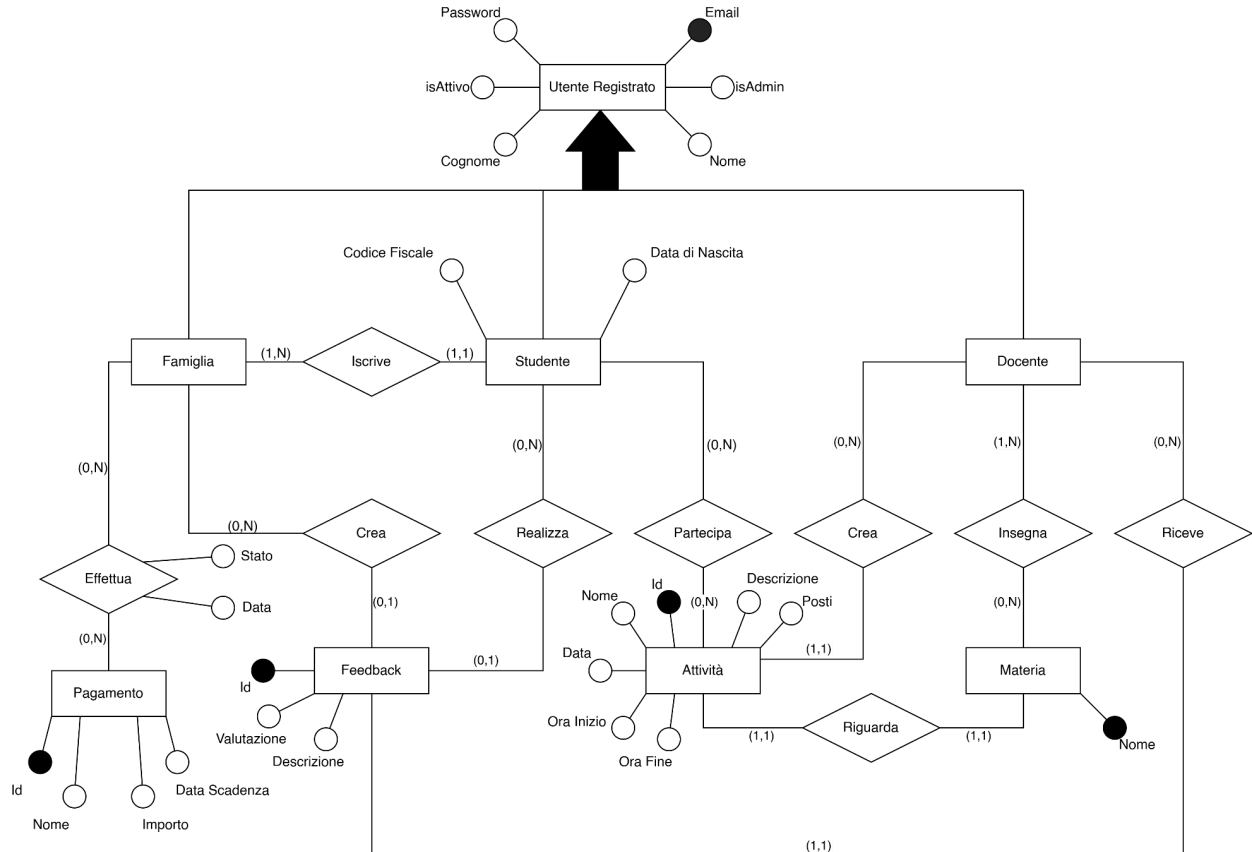
In particolare, tutte le entity identificate nella fase di Requirements Analysis sono state confermate senza necessità di aggiunte o rimozioni. Il motivo principale di questa scelta risiede nel fatto che la struttura individuata rispecchiava già in maniera ottimale le necessità funzionali, evitando ridondanze e mantenendo un elevato livello di chiarezza semantica.

Inoltre, non sono state introdotte nuove entity né eliminate componenti, poiché le funzionalità previste dalla release attuale non richiedevano estensioni o semplificazioni del modello. Il diagramma è stato quindi mantenuto nella sua forma originaria, limitandosi, ove necessario, a una riorganizzazione formale e a un affinamento delle relazioni per migliorarne la leggibilità, senza alterarne la struttura concettuale.





ER: Schema ER del database



Dizionario dei dati

Nel modello ER è stata modellata l'entità *UtenteRegistrato* come generalizzazione delle entità *Studente*, *Docente* e *Famiglia*. Questa scelta evidenzia formalmente che questi attori condividono un sottoinsieme di attributi comuni e rappresentano diverse specializzazioni dello stesso concetto di utente del sistema.

Per la traduzione dello schema concettuale in schema logico relazionale, abbiamo scelto di adottare una strategia di **Mapping Orizzontale**

Questa soluzione ci permette di eliminare la necessità di effettuare operazioni di JOIN con la tabella padre ogni volta che si recuperano i dati di un utente specifico, migliorando le prestazioni di lettura e semplificando la struttura delle query per le operazioni che coinvolgono solo una specifica tipologia di utente.



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

Di seguito si mostrano gli attributi per ogni entità individuata.

Nome Entità	Famiglia		
Descrizione	Contiene i dati relativi ad una famiglia		
Nome campo	Tipo	vincolo di chiave	Altri vincoli
email	Varchar(300)	Primary Key	NOT NULL
nome	Varchar(40)		NOT NULL
cognome	Varchar(40)		NOT NULL
password	Byte(32)		NOT NULL
isAttivo	Boolean		NOT NULL
isAdmin	Boolean		NOT NULL

Nome Entità	Studente		
Descrizione	Contiene i dati relativi ad uno studente		
Nome campo	Tipo	vincolo di chiave	Altri vincoli
email	Varchar(300)	Primary Key	NOT NULL
nome	Varchar(40)		NOT NULL
cognome	Varchar(40)		NOT NULL
password	Byte(32)		NOT NULL
codiceFiscale	Varchar(16)		NOT NULL
dataNascita	Date		NOT NULL
isAttivo	Boolean		NOT NULL
isAdmin	Boolean		NOT NULL
famiglia	Varchar(300)	Foreign Key	NOT NULL



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

Nome Entità	Docente		
Descrizione	Contiene i dati relativi ad un docente		
Nome campo	Tipo	vincolo di chiave	Altri vincoli
email	Varchar(300)	Primary Key	NOT NULL
nome	Varchar(40)		NOT NULL
cognome	Varchar(40)		NOT NULL
password	Byte(32)		NOT NULL
isAttivo	Boolean		NOT NULL
isAdmin	Boolean		NOT NULL

Nome Entità	Attività		
Descrizione	Contiene i dati relativi ad una attività		
Nome campo	Tipo	vincolo di chiave	Altri vincoli
Id_attività	Int(32)	Primary Key	NOT NULL
docente	Varchar(300)	Foreign Key	NOT NULL
materia	Varchar(300)	Foreign Key	NOT NULL
nome	Varchar(40)		NOT NULL
descrizione	Varchar(300)		NOT NULL
data	Date		NOT NULL
oraInizio	Time		NOT NULL
oraFine	Time		NOT NULL
posti	Int(32)		NOT NULL



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

Nome Entità	Materia		
Descrizione	Contiene i dati relativi ad una materia		
Nome campo	Tipo	vincolo di chiave	Altri vincoli
Nome	Varchar(40)	Primary Key	NOT NULL

Nome Entità	Feedback		
Descrizione	Contiene i dati relativi ad un feedback		
Nome campo	Tipo	vincolo di chiave	Altri vincoli
Id_feedback	int(32)	Primary Key	NOT NULL
Famiglia	Varchar(300)	Foreign Key	
Studente	Varchar(300)	Foreign Key	
Docente	Varchar(300)	Foreign Key	NOT NULL
valutazione	int(2)		NOT NULL
descrizione	Varchar(400)		

Nome Entità	Pagamento		
Descrizione	Contiene i dati relativi ad un pagamento		
Nome campo	Tipo	vincolo di chiave	Altri vincoli
Id_pagamento	int(32)	Primary key	NOT NULL
nome	Varchar(40)		NOT NULL
dataScadenza	Date		NOT NULL
importo	Double(32)		NOT NULL



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

Nome Entità	Effettua		
Descrizione	Contiene i dati relativi al pagamento effettuato		
Nome campo	Tipo	vincolo di chiave	Altri vincoli
Id_pagamento	int(32)	Primary Key Foreign Key	NOT NULL
famiglia	Varchar(300)	Primary Key Foreign Key	NOT NULL
data	Date		NOT NULL
stato	Varchar(15)		NOT NULL

Nome Entità	Partecipa		
Descrizione	Contiene i dati relativi agli studenti che partecipano alle attività		
Nome campo	Tipo	vincolo di chiave	Altri vincoli
Studente	Varchar(300)	Primary Key Foreign Key	NOT NULL
Attività	Int(32)	Primary Key Foreign Key	NOT NULL

Nome Entità	Insegna		
Descrizione	Contiene i dati relativi ai docenti che insegnano delle materie		
Nome campo	Tipo	vincolo di chiave	Altri vincoli
Docente	Varchar(300)	Primary Key Foreign Key	NOT NULL
Materia	Varchar(300)	Primary Key Foreign Key	NOT NULL



2.5 Controllo degli accessi e sicurezza

Oggetti	Attori	Studente	Docente	Famiglia
Autenticazione		Login Logout	Login Logout	Login Logout
Gestione Utente				RegistrazioneStudente
Gestione Attività		VisualizzazioneAttività IscrizioneAttività	VisualizzazioneAttività CreazioneAttività ModificaAttività EliminazioneAttività	
Gestione Feedback		InserimentoFeedback		InserimentoFeedback
Gestione Pagamenti				VisualizzazionePagament i EffettuaPagamento



Attori	Amministratore	Ospite
Oggetti		
Autenticazione		
Gestione Utenze	ModificaStatoUtenza	RegistrazioneDocente RegistrazioneFamiglia
Gestione Attività		
Gestione Feedback		
Gestione Pagamenti		

2.6 Controllo globale del software

Il sistema *Il Faro dello Studio* è una web application interattiva, nella quale ogni funzionalità viene attivata in seguito a un'azione eseguita dall'utente tramite l'interfaccia grafica. Quando l'utente seleziona una specifica operazione, l'interfaccia invoca il corrispondente componente di controllo, generando un evento che viene gestito dal relativo handler.

L'handler indirizza quindi il flusso verso il sottosistema dedicato alla logica di controllo, che a sua volta coordina le chiamate ai servizi responsabili dell'elaborazione applicativa.

Per queste ragioni, il sistema adotta un meccanismo di gestione del flusso di esecuzione basato su eventi (event-driven), soluzione particolarmente adatta alla natura interattiva delle applicazioni web.



2.7 Condizioni limite

Nel presente paragrafo verranno presentate le boundary conditions inerenti all'avvio del sistema, spegnimento del sistema, fallimento del sistema ed errore di accesso ai dati persistenti.

Identificativo	UCBC_1 – Avvio del sistema	Data	22/11/2025
		Versione	1.0
		Autori	Salvatore Amoroso
Descrizione	Questo Use Case permette l'avvio del sistema		
Attore principale	Amministratore		
Attori secondari	NA		
Entry condition	L'Amministratore accede al Server		
Exit condition On success	Il sistema viene avviato correttamente		
Exit condition On failure	Il sistema non viene avviato		
Flusso di eventi principale			
1	Amministratore	Esegue sulla macchina il comando che avvia il sistema.	
2	Sistema	Verifica la sanità dei dati persistenti e, se sani, rende disponibili i suoi servizi e rende le sue funzionalità disponibili agli utenti.	
Flusso di eventi alternativo: i dati persistenti sono danneggiati			



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

2.1	Sistema	Notifica l'Amministratore di problemi ai dati persistenti e non effettua l'avvio.
2.2	Amministratore	Corregge i dati persistenti
2.3	Amministratore	Esegue il passaggio 1 del flusso principale

Identificativo	UCBC_2 – Spegnimento del sistema	Data	22/11/2025
		Versione	1.0
		Autori	Salvatore Amoroso
Descrizione	Questo Use Case permette lo spegnimento del sistema		
Attore principale	Amministratore		
Attori secondari	NA		
Entry condition	L'Amministratore accede al Server AND Il Sistema è stato avviato AND Il Sistema è in esecuzione		
Exit condition On success	Il sistema viene spento correttamente		
Exit condition On failure	Il sistema non viene spento		
Flusso di eventi principale			



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

1	Amministratore	Invia un segnale di spegnimento al Sistema.
2	Sistema	Controlla che non ci siano connessioni ancora aperte da o verso l'esterno e, se non ci sono, termina l'esecuzione del sistema.
Flusso di eventi alternativo: ci sono connessioni ancora aperte		
2.1	Sistema	Notifica all'Amministratore che ci sono ancora connessioni aperte verso l'esterno.
2.2	Sistema	Attende una quantità di tempo per rispondere a eventuali richieste dall'esterno, non generando nuove connessioni se non allo scopo di rispondere a richieste già in corso.
2.3	Sistema	Controlla che non ci siano connessioni ancora aperte da o verso l'esterno e, se non ci sono, termina l'esecuzione del sistema.
2.4	Sistema	Notifica l'Amministratore dell'avvenuto spegnimento del sistema.
Flusso di eventi alternativo: ci sono connessioni ancora aperte		
3.1	Sistema	Recide le connessioni verso l'esterno.
3.2	Sistema	Notifica l'Amministratore dell'avvenuto spegnimento del sistema e del numero di connessioni recise.

Identificativo	UCBC_3 – Fallimento del sistema	Data	22/11/2025
		Versione	1.0



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

		Autori	Aniello Cristiano Ambrunzo
Descrizione	Questo Use Case definisce il comportamento del Sistema in caso di fallimento		
Attore principale	Amministratore		
Attori secondari	NA		
Entry condition	Il Sistema viene terminato inaspettatamente		
Exit condition On success	Il sistema viene riavviato correttamente		
Exit condition On failure	Il sistema non viene riavviato		
Flusso di eventi principale			
1	Amministratore	Include lo Use Case UCBC_1	

Identificativo	UCBC_4 – Errore di Accesso ai Dati Persistenti	Data	22/11/2025
		Versione	1.0
		Autori	Aniello Cristiano Ambrunzo
Descrizione	Questo Use Case descrive il comportamento del sistema qualora fosse impossibile accedere ai dati persistenti o questi risultassero corrotti.		



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

Attore principale		Amministratore
Attori secondari		NA
Entry condition		Il Sistema non può accedere ai dati persistenti. OR I dati persistenti risultano corrotti.
Exit condition On success		Il Sistema riprende il normale funzionamento
Exit condition On failure		Il Sistema non riprende il normale funzionamento
Flusso di eventi principale		
1	Sistema	Notifica l'Amministratore dell'impossibilità di accedere ai dati persistenti.
2	Sistema	Cessa di processare eventuali richieste dall'esterno e risponde a tutte le richieste con un messaggio di errore.
3	Amministratore	Include lo Use Case UCBC_2
4	Amministratore	Ripristina l'accessibilità o la sanità dei dati persistenti.
5	Amministratore	Include lo Use Case UCBC_1



2.8 Design Patterns

1. Facade Pattern

Utilizzo: Il **Facade Pattern** viene introdotto per isolare e coordinare il processo di business dell'iscrizione dello studente ad un'attività (**caso d'uso UC_GA_6**), che richiede l'interazione tra tre domini distinti: **Utenze**, **Attività** e **Pagamenti**. Invece di delegare questo coordinamento alla singola classe service (**ActivitiesService**), si crea un punto d'ingresso specializzato.

Obiettivi: gli obiettivi legati a questa scelta sono:

- Permettere di riportare **ActivitiesService** alla sua responsabilità primaria, ossia la gestione del ciclo di vita delle attività fornite dal doposcuola.
- Ridurre l'accoppiamento tra le singole classi del livello di business. Senza l'implementazione del Facade Pattern, la classe **ActivitiesService** presenterebbe un forte accoppiamento con le classi **UsersService** e **PaymentsService**.

Implementazione: si articola nelle seguenti componenti:

1. La classe **IscrizioneFacade** detiene i riferimenti privati verso i tre sottosistemi (**ActivitiesService**, **UsersService**, **PaymentsService**) e fornisce un metodo unificato **iscriviStudenteAdAttivita**.
2. Le classi service si occupano rispettivamente di:
 - a. **UsersService** fornisce il metodo per il recupero dei dati dello studente.
 - b. **PaymentsService** fornisce la logica di verifica della situazione debitoria.
 - c. **ActivitiesService** fornisce esclusivamente i metodi per visualizzare l'attività e persistere l'iscrizione finale.
3. La classe **ActivitiesController** diventerà il client della classe **IscrizioneFacade**



2. Observer Pattern

Utilizzo: L'**Observer Pattern** viene applicato al caso d'uso **Creazione Attività (UC_GA_1)**. Viene attivato ogni volta che un Docente crea una nuova attività all'interno del sistema. L'attivazione del pattern avviene immediatamente dopo il salvataggio persistente dell'attività nel database, fungendo da "ponte" tra la logica di business e il sistema di comunicazione verso l'esterno.

Obiettivo: L'obiettivo principale è garantire il disaccoppiamento logico tra il servizio che crea l'attività (**ActivitiesService**) e i meccanismi di notifica. Attraverso una dipendenza "uno-a-molti", il sistema permette di:

- Notificare istantaneamente tutti gli studenti della disponibilità di una nuova attività.
- Aggiungere in futuro nuovi canali di comunicazione (es. notifiche Push) senza dover modificare il codice sorgente del metodo **creaAttivita**.
- Assicurare che la notifica avvenga solo se l'attività è stata validata e salvata con successo.

Implementazione: si articola nelle seguenti componenti:

4. La classe **ActivitiesService** assume il ruolo di **Subject**. Essa mantiene una lista privata di osservatori registrati e fornisce i metodi per gestire le sottoscrizioni. Una volta completato il metodo **creaAttivita**, invoca internamente un metodo **notifyObservers()**.
5. L'interfaccia **NotificaObserver** che espone il metodo astratto **update(Attivita nuovaAttivita)**.
6. La classe concreta **EmailNotificationService** che implementa l'interfaccia **NotificaObserver**. All'interno del metodo **update**, il servizio recupera la lista degli studenti tramite lo **UsersService**, estrae i loro indirizzi email e procede all'invio massivo dei messaggi informativi utilizzando i dati della nuova attività.



3 Servizi dei sottosistemi

In questa sezione vengono descritti i servizi di ogni sottosistema precedentemente elencati.

Sottosistema Autenticazione

Servizio	Descrizione	Interfaccia
Login	Questa funzionalità permette di registrare lo studente alla piattaforma	AuthService
Logout	Questa funzionalità permette di registrarsi sulla piattaforma come docente	AuthService

Sottosistema Gestione Utenze

Servizio	Descrizione	Interfaccia
Gestione Stato Utenza	Questa funzionalità permette di attivare o disattivare un'utenza registrata	UserService
Registrazione studente	Questa funzionalità permette di registrare lo studente alla piattaforma	UserService



Laurea Triennale in Informatica-Università di Salerno
Corso di *Ingegneria del Software* 2025/2026
Prof. Gravino C.

Registrazione docente	Questa funzionalità permette di registrarsi sulla piattaforma come docente	UserService
Registrazione famiglia	Questa funzionalità permette di registrarsi sulla piattaforma come famiglia	UserService

Sottosistema Gestione Attività

Servizio	Descrizione	Interfaccia
Creazione attività	Questa funzionalità permette di creare una nuova attività	ActivitiesService
Modifica attività	Questa funzionalità permette di modificare un'attività esistente	ActivitiesService
Eliminazione attività	Questa funzionalità permette di eliminare un'attività esistente	ActivitiesService
Visualizzazione calendario attività	Questa funzionalità permette di visualizzare il calendario delle attività	ActivitiesService



Iscrizione ad attività	Questa funzionalità permette di iscriversi ad un'attività	ActivitiesService
-------------------------------	---	-------------------

Sottosistema Gestione Feedback

Servizio	Descrizione	Interfaccia
Inserimento feedback	Questa funzionalità permette di inviare un feedback ad un docente	FeedbackService

Sottosistema Gestione Pagamenti

Servizio	Descrizione	Interfaccia
Visualizzazione pagamenti	Questa funzionalità permette di visualizzare i pagamenti	PaymentsService
Effettuare pagamento	Questa funzionalità permette di effettuare un pagamento	PaymentsService



4 Glossario

- **Back-end** La parte "invisibile" del software che gira sul server e gestisce il funzionamento del sistema.
- **COTS (Commercial Off The Shelf)** Componenti software già pronti e disponibili sul mercato, usati per non dover creare tutto da zero.
- **CRUD** Acronimo per le 4 operazioni fondamentali sui dati: **C**rea, **L**eggi, **A**ggiorna, **C**ancella.
- **DAO (Data Access Object)** La parte di codice che si occupa specificamente di leggere e scrivere i dati nel database.
- **DBMS (Database Management System)** Il programma che organizza e conserva i dati in modo sicuro (nel documento è MySQL).
- **Entity (Entità)** Rappresentazione digitale di un oggetto reale (come "Studente" o "Attività") i cui dati vengono salvati.
- **Event-driven** Uno stile di funzionamento dove il programma reagisce alle azioni dell'utente (es. click del mouse) invece di seguire un ordine fisso.
- **Front-end** L'interfaccia grafica (pagine web) che l'utente vede e con cui interagisce.
- **Overhead** Il lavoro o le risorse extra (come memoria o tempo) che il computer "spreca" per gestire controlli e sicurezza.
- **UML** Un linguaggio standard fatto di schemi e diagrammi per descrivere visivamente come è fatto il software.